Architecture Decision Report

ApontaUA — Collaborative Academic Document Management System

M2 — Lifecycle Architecture

Team: Daniel Simbe, Anderson Tembe, Giovanni Santos,
Felizardo Samuel, Marcelo Junior and Keegan Azevedo
Supervision: João Paulo Barraca and Catarina Silva

Status: Proposed architecture for team and supervisor validation

1. Purpose and scope

This report records the proposed architecture and principal technology decisions for the ApontaUA MVP. ApontaUA is a collaborative academic document management system intended to centralize, organize, discover, share and validate academic materials produced by students and teachers.

The architecture prioritizes a manageable implementation for a six-person student team, clear separation of responsibilities, testability and the ability to evolve after the MVP. Conditional and future decisions must be confirmed against the final requirements and available infrastructure.

2. Architecture overview

The system uses a modular monolith: one FastAPI backend application organized into functional modules and a separate React frontend. PostgreSQL stores structured application data. Original uploaded files are kept in persistent file storage. A separate Python worker and Redis-backed task queue are proposed if OCR and document processing must run asynchronously in the MVP.

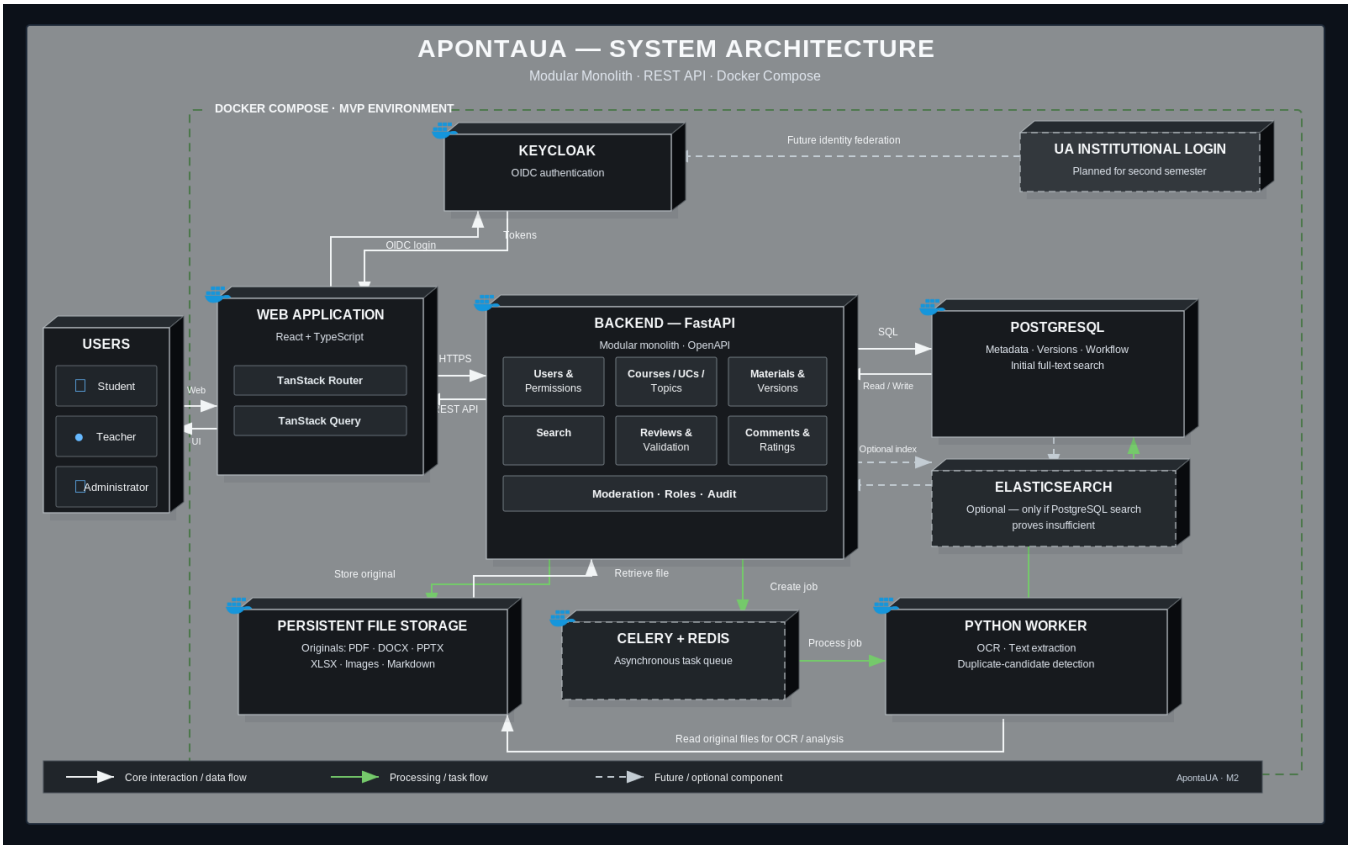


Figure 1. Dark technical architecture diagram. Dashed outlines and connectors indicate future or optional components.

3. Architecture principles

- Modular monolith first — one backend deployment with well-defined functional modules; no microservices unless a demonstrated need emerges.
- API boundary — the frontend communicates with the backend through HTTPS and a documented REST API. Business rules and authorization remain server-side.
- Relational source of truth — PostgreSQL is authoritative for structured data and workflow state.
- Original-file preservation — keep submitted originals separate from metadata and extracted text; derived representations do not replace the original.
- Incremental complexity — start with PostgreSQL full-text search and add infrastructure only when requirements or measurements justify it.
- Security by default — validate tokens, permissions, uploads and inputs on the server; frontend visibility is not authorization.

4. Architecture decisions

ADR-01 — Frontend framework

Status: Accepted proposal

Decision. Use React with TypeScript and Vite. Use TanStack Router for client-side routing and TanStack Query for API data fetching, caching and mutation state.

Rationale. Fits a single-page application and supports a feature-oriented frontend without introducing a separate server-rendering framework.

ADR-02 — Backend framework and style

Status: Accepted proposal

Decision. Use Python with FastAPI as a modular monolith. Expose a REST API documented through OpenAPI. Separate routers, schemas, services and persistence concerns within business modules.

Rationale. A single backend deployment is simpler to develop, test and operate than multiple services, while module boundaries support maintainability.

ADR-03 — Authentication for MVP

Status: Accepted for initial phase

Decision. Use Keycloak as the initial identity provider with OpenID Connect (OIDC). The frontend follows the login flow and sends an access token to the API. FastAPI validates the token and applies application permissions.

Rationale. Enables a realistic authentication flow before institutional integration is available. Token validation and authorization must be implemented and tested.

ADR-04 — Institutional authentication

Status: Deferred to second semester

Decision. Plan integration with the University of Aveiro identity provider, subject to confirmation of the supported protocol and access conditions. Keep application authorization rules as independent as practical from the identity provider.

Rationale. Prevents the MVP from depending on institutional integration availability. The later integration is not guaranteed until UA requirements are confirmed.

ADR-05 — Primary database

Status: Accepted proposal

Decision. Use PostgreSQL for academic structure, materials, metadata, versions, comments, ratings, validation states, audit events and extracted text where appropriate.

Rationale. The domain contains related entities and workflow constraints that fit a relational model. PostgreSQL remains the source of truth.

ADR-06 — Search

Status: Accepted for MVP

Decision. Start with PostgreSQL full-text search and suitable indexes. Do not deploy Elasticsearch initially. Reconsider it only if measured search quality, scale or performance justify the additional component.

Rationale. Reduces deployment and synchronization complexity while supporting the initial search requirements.

ADR-07 — Original file storage

Status: Proposed

Decision. Store original uploads in a persistent volume. Store file identifiers, metadata, ownership and processing state in PostgreSQL. Define backup and restore procedures before the pilot.

Rationale. Separates binary files from structured records. Persistence must survive container recreation and be covered by backups.

ADR-08 — Asynchronous document processing

Status: Conditional proposal

Decision. If OCR and long-running extraction/similarity jobs are in the MVP, use a separate Python worker with Celery and Redis as the task broker. Persist task status, failures and results; allow safe retries.

Rationale. Keeps slow processing out of request-response paths. Confirm scope before introducing these operational components.

ADR-09 — Duplicate detection

Status: Proposed staged implementation

Decision. First detect exact file duplicates using a cryptographic hash. Then compare extracted text to identify candidate similar materials. Present candidates as warnings; do not automatically reject content solely because it is similar.

Rationale. Provides useful warnings incrementally and avoids treating complementary materials as duplicates.

ADR-10 — Editor and imported files

Status: Open implementation decision

Decision. Evaluate TipTap or ProseMirror for native content editing. Treat imported PDF, DOCX, PPTX, XLSX and image files as distinct from editor-native Markdown content. Define for each supported format whether the MVP stores, previews, indexes or edits it.

Rationale. An editor does not automatically convert every uploaded format into editable Markdown. The expected user experience must be explicit.

ADR-11 — Containers and CI

Status: Recommended

Decision. Use Docker Compose for local integration of the frontend, API, PostgreSQL and Keycloak. Add Redis and the worker only if required. Use GitHub Actions for automated checks and tests where feasible.

Rationale. Reproducible environments help a multi-person team integrate consistently. Grow the CI pipeline incrementally.

ADR-12 — Microservices

Status: Rejected for MVP

Decision. Do not split the core application into microservices. The document-processing worker, if adopted, is a separate process, not a separate business microservice.

Rationale. The current scope does not justify the deployment, networking, observability and consistency costs of independently deployed business services.

5. Main component responsibilities

- React frontend — user interface, navigation, forms, browsing, submission, search results and presentation of processing/review states.
- Keycloak — initial authentication and OIDC token issuance; not the source of truth for academic materials or workflow.
- FastAPI backend — REST endpoints, request validation, business rules, permission enforcement and orchestration of persistence and jobs.
- PostgreSQL — structured data, relationships, workflow state, versions, audit information and initial full-text search.
- Persistent file storage — original uploaded files; access is controlled and arbitrary filesystem paths are never exposed.

- Celery + Redis, if adopted — coordination and execution of long-running processing tasks; Redis is not the authoritative application database.
- Python worker, if adopted — OCR/text extraction, duplicate-candidate analysis and reporting processing status/results.
- Elasticsearch, future only — optional search index if PostgreSQL search proves insufficient; PostgreSQL remains the source of truth.

6. Proposed document-submission flow

1. The user submits a file or creates content through the frontend.
2. FastAPI authenticates the request, checks permissions and validates the supported format and upload limits.
3. The original file is saved to persistent storage; metadata and an initial processing state are recorded in PostgreSQL.
4. If asynchronous processing is enabled, the API enqueues a job. The worker extracts text and performs duplicate checks as applicable.
5. The worker records results and processing status. Failures are logged and can be retried without losing the original.
6. The frontend displays processing results and candidate duplicate warnings. Moderation, peer review and academic validation remain separate workflow decisions.

7. Security and reliability considerations

- Validate OIDC access tokens on the API side and enforce authorization for every protected operation.
- Apply role and resource-level permissions, including ownership checks and unit-specific validation rights where required.
- Validate upload size, file type and content; do not trust filenames or MIME types supplied by clients.
- Preserve original files; edits or replacements should create a new version and preserve history.
- Processing success must not automatically imply publication or academic validation.
- Define backup and restore procedures for PostgreSQL and uploaded files.
- Record relevant security and workflow events without logging passwords, access tokens or unnecessary personal data.
- Keep secrets out of source control and protect administrative interfaces.

8. Deployment view

For local development, Docker Compose can coordinate the frontend, FastAPI API, PostgreSQL and Keycloak. If asynchronous processing is in scope, add Redis and the Python worker. Persistent volumes are required for database data and uploaded files. Production hosting and institutional deployment details remain to be confirmed.

9. Decisions requiring confirmation

- Confirm whether OCR and asynchronous document processing are part of the MVP; if yes, confirm Celery + Redis.
- Confirm the exact file formats and maximum upload size supported by the MVP.
- Define how each format is previewed, indexed and, if applicable, edited.
- Specify initial roles and resource-level authorization rules.
- Confirm Keycloak realm/client configuration and the token validation approach for FastAPI.
- Define backup and restore expectations for database and file storage.
- Agree acceptance criteria for search, OCR and duplicate-detection quality.
- Confirm UA identity integration requirements before the second-semester implementation.

10. Summary

The proposed architecture is a React/TypeScript frontend backed by a modular FastAPI monolith, PostgreSQL and Keycloak for initial authentication. Original files are stored separately from structured metadata. PostgreSQL full-text search is the initial search solution. A Celery/Redis queue and Python worker are recommended only if long-running OCR and document-processing tasks are required in the MVP. Institutional authentication and Elasticsearch are deferred. This design keeps the first implementation manageable while providing clear paths for future extension.

References

- FastAPI documentation — https://fastapi.tiangolo.com/
- React documentation — https://react.dev/
- TanStack Router — https://tanstack.com/router/latest
- TanStack Query — https://tanstack.com/query/latest
- Keycloak documentation — https://www.keycloak.org/documentation
- PostgreSQL full-text search — https://www.postgresql.org/docs/current/textsearch-intro.html
- Celery documentation — https://docs.celeryq.dev/
- Docker Compose documentation — https://docs.docker.com/compose/